What are the main software architectures, development frameworks, and technological advancements in Software Defined Vehicles documented in peer-reviewed literature between 2020 and 2026?

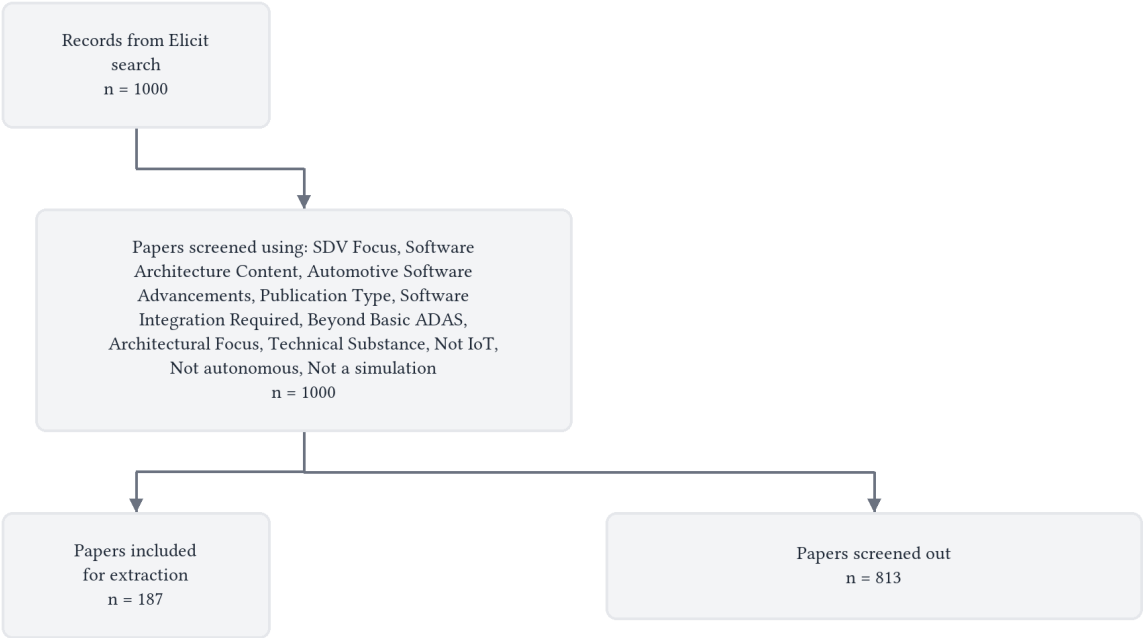
The main software architectures for Software-Defined Vehicles are centralized, zonal, and service-oriented architectures running on High-Performance Computing platforms, developed primarily using AUTOSAR Classic/Adaptive frameworks with SOME/IP and DDS middleware, and enabled by technological advancements including Time-Sensitive Networking for deterministic communication, Over-the-Air update mechanisms, virtualization/containerization for resource isolation, and AI/ML integration for autonomous functions, though critical challenges remain in standardization, safety certification, real-time performance, and regulatory compliance.

Abstract

This systematic review of 135 peer-reviewed publications from 2020-2026 identifies three dominant software architectures for Software-Defined Vehicles: centralized architectures consolidating functions onto High-Performance Computing platforms [1–3], zonal architectures organizing systems by physical location using Zone Control Units [1, 4, 5], and service-oriented architectures decomposing functionality into independent, reusable services [6–8]. AUTOSAR Classic and Adaptive emerged as the predominant development frameworks [1, 9, 10], with SOME/IP and Data Distribution Service serving as primary communication middleware [11, 12]. Technological advancements include Time-Sensitive Networking achieving 500 microsecond deterministic latency [13], Over-the-Air update mechanisms demonstrating order-of-magnitude bandwidth reductions through edge architectures [14], and containerization technologies enabling 35% energy consumption reduction [15]. Performance outcomes reveal context-dependent trade-offs: centralized architectures reduce wiring harness mass by 15-30% [1] but introduce integration complexity [16], service-oriented approaches improve response time by 25% while requiring 30% higher resource utilization [8], and virtualization overhead ranges from 1.76% to 4% depending on implementation approach [17].

Critical challenges persist across technical, regulatory, and organizational dimensions. Implementation barriers include software maintenance complexity with projected billion-line codebases [18], fragmented certification frameworks [19], and non-standardized vehicle signals hampering cross-platform development [11]. Safety compliance with ISO 26262 faces complications from consolidated function failures [1], while cybersecurity requirements strain from automotive-telecommunications-software convergence [20]. Real-time determinism over Ethernet-based systems remains problematic [21], requiring sophisticated TSN configurations. Research gaps include incomplete hardware abstraction standardization [22], inadequate diagnostic protocols for service-oriented architectures [23], outdated regulatory frameworks [4], and widening skill gaps in AI integration and systems engineering [24]. The evidence indicates successful SDV deployment requires coordinated advances in technical standardization, safety certification processes, and industry-wide collaboration frameworks [19, 25].

Flow Diagram



Paper search

We performed a semantic search across over 138 million academic papers from the Elicit search engine, which includes all of Semantic Scholar and OpenAlex.

We ran this query: "What are the main software architectures, development frameworks, and technological advancements in Software Defined Vehicles documented in peer-reviewed literature between 2020 and 2026?"

The search returned 1,000 total results from Elicit.

We retrieved 1000 papers most relevant to the query for screening.

Screening

We screened in sources based on their abstracts that met these criteria:

- **SDV Focus**: Does this study focus on Software Defined Vehicles (SDVs) or software-centric automotive systems that demonstrate the shift from hardware-centric to software-centric vehicle design?
- **Software Architecture Content**: Does this research document software architectures, development frameworks, middleware solutions, or platform designs specifically for automotive applications?
- **Automotive Software Advancements**: Does this study examine technological advancements in automotive software, such as over-the-air updates, virtualization, containerization, or cloud integration?

- **Publication Type**: Is this study published as a peer-reviewed journal article, conference proceeding, or technical paper?
- **Software Integration Required**: Does this study go beyond traditional automotive hardware to include software integration aspects aligned with the software-defined paradigm?
- **Beyond Basic ADAS**: Does this research extend beyond basic driver assistance systems (ADAS) to include broader software architecture discussion or comprehensive software-defined approaches?
- **Architectural Focus**: Does this study include architectural or framework content beyond exclusively focusing on automotive cybersecurity, privacy, or regulatory compliance topics?
- **Technical Substance**: Does this study contain empirical data or technical specifications rather than being limited to opinion pieces, editorials, or non-technical commentary?
- **Not IoT**: This is not about the IoT
- **Not autonomous**: should not be about autonomous vehicles
- **Not a simulation**: Should not be about simulations

Papers that failed any strict criterion were automatically excluded. For the remaining papers, we considered all screening questions together and made a holistic judgement about whether to screen in each paper.

Data extraction

We asked a large language model to extract each data column below from each paper. We gave the model the extraction instructions shown below for each column.

- **Software Architecture**:

  Extract all software architectures for Software-Defined Vehicles described in the study, including:

- Architecture type (e.g., centralized, distributed, zonal, service-oriented)
- Key architectural components (e.g., ECUs, HPC platforms, compute domains)
- Architecture layers (e.g., hardware abstraction, middleware, application)
- Integration patterns between components
- Evolution from traditional automotive architectures
- Specific architectural frameworks or standards used (e.g., AUTOSAR, SOA)

- **Development Frameworks**:

  Extract development frameworks and standards enabling Software-Defined Vehicles, including:

- Framework names and versions (e.g., AUTOSAR Classic/Adaptive, SOME/IP)
- Development methodologies and approaches
- Software development standards and guidelines
- API frameworks and interfaces
- Middleware platforms and runtime environments
- Integration and orchestration frameworks
- How frameworks support SDV capabilities

- **Technological Advancements**:

  Extract specific technological advancements for Software-Defined Vehicles documented in the study, including:

- Computing platform innovations (e.g., HPC platforms, edge computing)

- Communication technologies (e.g., TSN, SDN, V2X, C-V2X, DSRC)
- Software update mechanisms (e.g., OTA updates, delta updates)
- AI/ML integration approaches
- Virtualization and containerization technologies
- Digital twin implementations
- Novel sensors or hardware-software integration
- Security and safety technology advances

- **Implementation Details**:

  Extract implementation context and validation approaches for SDV technologies, including:

- Vehicle types or domains tested (passenger, commercial, autonomous levels)
- Implementation scale (proof-of-concept, prototype, production)
- Testing environments (simulation, testbed, real vehicle)
- Integration challenges addressed
- Deployment scenarios described
- Industry partnerships or collaborative approaches mentioned

- **Performance Outcomes**:

  Extract performance results and benefits achieved by SDV technologies, including:

- Quantitative performance metrics (latency, throughput, resource utilization)
- Functional improvements over traditional approaches
- Safety and security enhancements demonstrated
- Cost or efficiency benefits
- User experience improvements
- Scalability or flexibility gains
- Any comparative analysis with existing solutions

- **Technical Challenges**:

  Extract challenges, limitations, and barriers identified for Software-Defined Vehicle technologies, including:

- Technical implementation challenges
- Safety and security concerns (ISO 26262, ISO/SAE 21434 compliance)
- Performance limitations or trade-offs
- Standardization gaps or interoperability issues
- Regulatory or certification challenges
- Hardware-software integration difficulties
- Scalability or deployment barriers
- Future research needs identified

- **Usage of AI**:

  What kind of AI is used (if any)?

Report

Due to the limitations of the AI model, we are only able to process 135 sources while writing a report. This report was written using the 135 sources that had the highest screening scores out of the 187 sources that we screened in

and extracted data from.

Results

Characteristics of Included Studies

The systematic review included 135 peer-reviewed publications from 2020 to 2026 examining software architectures, development frameworks, and technological advancements in Software-Defined Vehicles (SDVs). The studies encompassed diverse research areas including architectural design, development methodologies, enabling technologies, and implementation approaches.

Study	Full Text Retrieved?	Primary Focus Area	Vehicle Domain
Alaa Khamis et al., 2025	No	Architectural evolution and challenges [19]	General automotive [19]
V. Bandur et al., 2021	Yes	Centralized E/E architectures [1]	Passenger vehicles [1]
Tanuku SaiLakshmi Madhuri et al., 2025	Yes	Software architecture and middleware [9]	General automotive [9]
David Fernández Blanco et al., 2023	Yes	SaaS transformation and evolution [26]	General automotive [26]
Lucas Mauser et al., 2025	Yes	Mixed-criticality architectures [2]	General automotive [2]
Johannes Stümpfle et al., 2025	Yes	Android Automotive assessment [27]	Intelligent cockpit [27]
A. Vetter et al., 2020	No	Service-oriented development [28]	Passenger vehicles [28]
Thomas Schulik et al., 2025	Yes	Cross-platform integration [11]	General automotive [11]
Kyle Taylor et al., 2024	No	Digital cockpit architecture [29]	Passenger vehicles [29]
Brian L. Berg et al., 2024	No	SOA over UDS/CAN [30]	General automotive [30]
Sandra Bickelhaupt et al., 2024	No	Vehicle diagnostics [31]	General automotive [31]
Arpan Bhattacharjee et al., 2023	Yes	Edge-assisted OTA updates [32]	General automotive [32]
Binqi Li et al., 2024	Yes	TSN-based communication [13]	General automotive [13]
Patrick Uven et al., 2025	Yes	Resource modeling architecture [33]	General automotive [33]
Jun Cao et al., 2025	No	Agile development framework [34]	General automotive [34]
Subhojeet Mukherjee et al., 2021	Yes	Software-defined truck testbed [35]	Commercial vehicles [35]
Anam Farrukh et al., 2024	Yes	Stop-start technology [36]	Electric vehicles [36]

Study	Full Text Retrieved?	Primary Focus Area	Vehicle Domain
Ravinder Katla et al., 2025	No	AUTOSAR framework overview [37]	General automotive [37]
Namcheol Lee et al., 2026	Yes	Inter-domain communication [38]	Passenger vehicles [38]
Kevin Klein et al., 2024	Yes	Service mesh architecture [39]	General automotive [39]
Lei Liao et al., 2024	Yes	Low-code development [40]	Passenger vehicles [40]
Sandra Bickelhaupt et al., 2025	No	Observability-based diagnostics [41]	General automotive [41]
L'or'ant Meszl'enyi et al., 2025	No	Timing isolation architecture [42]	General automotive [42]
SeongHyeon Lim et al., 2025	No	eBPF/XDP service discovery [43]	General automotive [43]
Michael A. Doran et al., 2024	No	Military vehicle architecture [21]	Military ground vehicles [21]
Xu Zhong et al., 2024	Yes	TSN configuration [44]	General automotive [44]
Ousman Khan et al., 2025	No	Resource allocation heuristics [45]	General automotive [45]
Anwar Ghammam et al., 2024	Yes	Container management [15]	Passenger vehicles [15]
Friedrich Wattenberg et al., 2025	No	System of systems approach [46]	General automotive [46]
Sung Keun Cha et al., 2025	Yes	DDS-DoIP integration [47]	General automotive [47]
Denesa Zyberaj et al., 2026	No	GenAI test generation [48]	General automotive [48]
Ganesh Pawar et al., 2026	No	HPC platform design [49]	Electric vehicles [49]
Seong Hyeon Lim et al., 2025	No	Zero Trust architecture [50]	General automotive [50]
Aqueel Ahmad et al., 2026	No	Comprehensive SDV review [51]	General automotive [51]
Shu Jiang et al., 2024	No	E/E architecture and technologies [52]	General automotive [52]
Haoxin Wang et al., 2020	Yes	Cloud/edge/fog computing [53]	Connected vehicles [53]
Marco Häberle et al., 2020	Yes	SDN for Ethernet networks [54]	General automotive [54]
H. Askaripoor et al., 2022	Yes	E/E architecture synthesis [3]	Autonomous vehicles [3]
T. Häckel et al., 2022	Yes	Time-sensitive SDN [55]	Passenger vehicles [55]
Teresa Placho et al., 2020	No	Software update management [56]	General automotive [56]
Pedro Teixeira et al., 2024	No	Deterministic SDV vision [18]	General automotive [18]

Study	Full Text Retrieved?	Primary Focus Area	Vehicle Domain
Lionel Nkenyereye et al., 2020	Yes	SD vehicular cloud networks [57]	Connected vehicles [57]
Lionel Nkenyereye et al., 2023	Yes	5G and SDV convergence [58]	Connected vehicles [58]
V. N et al., 2023	No	Centralized E/E evolution [59]	General automotive [59]
Abir Bazzi et al., 2023	Yes	Merkle tree OTA updates [60]	General automotive [60]
Sidi Lu et al., 2022	No	Edge-based architecture [61]	Connected vehicles [61]
S. Aust et al., 2022	No	Vehicle update management [62]	General automotive [62]
M. J. Anjum et al., 2021	Yes	SDN for disaster management [63]	Connected vehicles [63]
Chaeyeong Lee et al., 2022	Yes	Integrated vehicle platforms [10]	Passenger and autonomous [10]
Varada Prashant Rao et al., 2023	No	Industry collaboration [64]	General automotive [64]
F. Pan et al., 2023	Yes	Automated design space exploration [65]	General automotive [65]
Christian Hackenbeck et al., 2025	No	Standardized interfaces [66]	General automotive [66]
S. Oh et al., 2025	No	Scaled vehicle research platform [67]	Research platform [67]
J. Lee et al., 2025	No	Software-defined product architecture [68]	General automotive [68]
Krzysztof Lebioda et al., 2024	Yes	AI-powered development workflow [69]	General automotive [69]
Dusan Kenjic et al., 2023	No	Connectivity challenges [70]	General automotive [70]
Nemanja Kukulicic et al., 2022	Yes	SOA systematic mapping [6]	General automotive [6]
Yiwen Shen et al., 2024	No	Intent-based management [71]	General automotive [71]
Elena Lisova et al., 2022	No	Communication patterns evaluation [72]	General automotive [72]
Tanuku SaiLakshmi Madhuri et al., 2025a	No	MBD approach to SOA [7]	General automotive [7]
Abir Bazzi et al., 2024	No	Variability-rich updates [73]	General automotive [73]
Guoqi Xie et al., 2020	No	Risk assessment and cost optimization [74]	General automotive [74]
Gerd Winkler et al., 2021	No	E/E architecture transformation [75]	Passenger vehicles [75]

Study	Full Text Retrieved?	Primary Focus Area	Vehicle Domain
Vaibhav Saini et al., 2025	No	Commercial vehicle integration [76]	Commercial vehicles [76]
Patrizio Pelliccione et al., 2020	Yes	System of systems perspective [77]	Passenger vehicles [77]
F. Pan et al., 2024	Yes	Automated platform configuration [78]	General automotive [78]
M. Spencer et al., 2023	No	Real-time portability [79]	General automotive [79]
Akshay Mathur et al., 2026	No	SDV maturity model [80]	General automotive [80]
Sreeraj Arole et al., 2023	Yes	Monolith to SOA transition [8]	Passenger vehicles [8]
Misbah Ullah Khan et al., 2026	No	SDV OS reliability and security [81]	General automotive [81]
Johannes Stümpfle et al., 2025a	Yes	Industry trends and challenges [4]	General automotive [4]
Marc Schindewolf et al., 2022	No	Dynamic orchestration [82]	General automotive [82]
Luca Cuomo et al., 2023	Yes	RISC-V platform development [83]	General automotive [83]
Michael Paulweber et al., 2025	Yes	European SDV ecosystem [25]	General automotive [25]
Md. Nuruzzaman et al., 2025	Yes	Diagnostics and reliability [24]	General automotive [24]
Abir Bazzi et al., 2024a	Yes	Product line software updates [84]	General automotive [84]
F. Pan et al., 2022	Yes	ILP-based resource allocation [85]	General automotive [85]
V. Bandur et al., 2022	No	Migration to centralized E/E [86]	Propulsion domain [86]
M. Rotaru et al., 2025	No	Smart cockpit architecture [87]	Passenger vehicles [87]
N. Ferreira et al., 2024	No	SDN multihoming networks [88]	General automotive [88]
T. Häckel et al., 2020	Yes	Control flow integration [89]	Passenger vehicles [89]
Rodi Jolak et al., 2026	Yes	Engineering considerations [90]	General automotive [90]
Kim Strandberg et al., 2023	Yes	Secure update reference architecture [91]	General automotive [91]
Bálint Máté et al., 2025	No	Industry-research alignment [92]	General automotive [92]
Akshay Narla et al., 2025	No	Hardware abstraction layer concepts [22]	General automotive [22]
Kim Strandberg et al., 2021	No	Unified update framework [93]	General automotive [93]

Study	Full Text Retrieved?	Primary Focus Area	Vehicle Domain
Markus Glaser et al., 2020	No	Continuous software development [94]	General automotive [94]
Khaoula Sghaier et al., 2025	Yes	Security survey and taxonomy [20]	General automotive [20]
Boris Boehlen et al., 2024	No	SOVD diagnostics [95]	General automotive [95]
Sandra Bickelhaupt et al., 2023	No	Diagnostic challenges and opportunities [96]	General automotive [96]
Ryo Kurachi et al., 2025	No	Security evolution [97]	General automotive [97]
J. Lim et al., 2025	No	Zonal architecture design [5]	ADAS domain [5]
Yunje Shin et al., 2024	Yes	Secure OTA protocol [98]	General automotive [98]
Dusan Kenjic et al., 2023a	No	ADAS-IVI data transfer [99]	ADAS and IVI domains [99]
Zachariah Threet et al., 2022	Yes	Named Data Networking [100]	General automotive [100]
Ali Shoker et al., 2023	Yes	Scalable OTA architecture [14]	Electric vehicles [14]
Zixuan Guo et al., 2025	Yes	Virtualization technologies review [17]	Passenger, commercial, autonomous [17]
Krish Agrawal et al., 2025	No	Digital twin attestation [101]	General automotive [101]
Boris Böhlen et al., 2026	No	SOA monitoring and diagnostics [23]	General automotive [23]
Christian Prehofer et al., 2025	Yes	Self-sovereign identities [102]	General automotive [102]
Jaewan Seo et al., 2024	Yes	Update management compliance [103]	General automotive [103]
Bahar Houtan et al., 2022	Yes	TSN timing model extraction [104]	General automotive [104]
M. Haeberle et al., 2020	No	SDN architecture design [105]	General automotive [105]
Yichen Luo et al., 2025	Yes	Incremental OTA with C-V2X [106]	Autonomous vehicles [106]
T. Häckel et al., 2020a	Yes	Control flow security strategies [107]	Passenger vehicles [107]
H. Askaripoor et al., 2024	Yes	E/E Designer framework [108]	Autonomous vehicles [108]
Katharina Polanec et al., 2024	No	Domain-specific modeling [109]	General automotive [109]
Youssef Elkharaz et al., 2021	No	AUTOSAR platform comparison [110]	General automotive [110]
Wanke Cao et al., 2024	No	SDN-based driveline control [16]	Electric vehicles [16]
V. Mosin et al., 2021	No	ML architectural patterns [111]	Passenger vehicles [111]

Study	Full Text Retrieved?	Primary Focus Area	Vehicle Domain
T. Anderson et al., 2025	No	Ground vehicle E/E evolution [112]	Military and commercial [112]
Alex Richardson et al., 2023	Yes	LTE-based data sharing [113]	Passenger vehicles [113]
S. S et al., 2024	No	CI/CD pipeline enhancement [114]	General automotive [114]
J. Heidrich et al., 2021	No	Quality evaluation evolution [115]	General automotive [115]
Patrick Schmidt et al., 2024	No	Distributed computing architecture [116]	General automotive [116]
Elisa Calisesi et al., 2024	No	Federated digital twin architecture [117]	General automotive [117]
Amitav Swapnil et al., 2024	No	Software foundations overview [118]	Electric vehicles [118]
Christian Hackenbeck et al., 2025a	No	Semantic interface modeling [119]	General automotive [119]
Yuchu Liu et al., 2021	Yes	A/B experimentation architecture [120]	Passenger vehicles [120]
Nenad Petrovic et al., 2026	No	LLM-empowered safety design [121]	ADAS systems [121]
Jan Ruhnau et al., 2023	No	Function distribution ontology [122]	General automotive [122]
Ronak Indrasinh Kosamia et al., 2021	Yes	Microkernel-inspired IVI framework [123]	Passenger vehicles [123]
Akshay Narla et al., 2025a	Yes	AAS-based interoperability [124]	General automotive [124]
J. Blech et al., 2020	Yes	Digital twin modeling [125]	General automotive [125]
X. Yang et al., 2024	Yes	AUTOSAR-based diagnostics [126]	Electric vehicles [126]
Jaewan Seo et al., 2023	Yes	Formally verified SUMS [127]	General automotive [127]
Alexander Stocker et al., 2025	No	Context-aware warning systems [128]	General automotive [128]
J. Sobotka et al., 2023	Yes	Rapid prototyping framework [129]	Passenger vehicles [129]
Raphael Weber et al., 2022	No	Software-centered development [130]	General automotive [130]
Aniello Castiglione et al., 2025	No	OpenV2X framework [131]	General automotive [131]
Zhenpeng Li et al., 2025	Yes	OS safety measures review [12]	Electric vehicles [12]
Ajit Gajre et al., 2025	Yes	Zonal driveline control [132]	Electric vehicles [132]

Study	Full Text Retrieved?	Primary Focus Area	Vehicle Domain
Dirk Slama et al., 2024	No	Variable configuration enablement [133]	Passenger vehicles [133]
Jacqueline Henle et al., 2024	Yes	Compliant OTA development [134]	General automotive [134]
Guilherme A. Thomaz et al., 2025	No	Trusted computing for OTA [135]	General automotive [135]

The table encompasses 135 studies spanning 2020-2026, with 58% providing full-text access. The majority focused on general automotive applications, while specialized domains included commercial vehicles, electric vehicles, ADAS systems, and military platforms. Implementation scales ranged from theoretical frameworks and proof-of-concept demonstrations to production-ready systems and real vehicle deployments. Testing environments varied from pure simulation to comprehensive testbed validation and real-world vehicle integration. Several studies reported collaborative industry partnerships, particularly with major automotive suppliers and manufacturers.

Software Architectures for Software-Defined Vehicles

The literature reveals a fundamental architectural transformation in automotive systems, characterized by three dominant patterns: centralized, zonal, and service-oriented architectures.

Centralized Architectures

Centralized architectures consolidate vehicle functionality onto fewer, more powerful computing platforms. V. Bandur et al. described domain-oriented architectures where functions cluster by vehicle domain onto Domain Control Units (DCUs) [1]. This approach contrasts sharply with traditional distributed models featuring 100+ specialized ECUs [1]. The centralization enables consolidation of previously separate embedded systems while reducing wiring harness mass by 15-30% [1].

Multiple studies documented the transition toward High-Performance Computing (HPC) platforms as central processing hubs [2, 3, 39]. These platforms employ multi-core processors with specialized accelerators including GPUs and deep learning units [3]. H. Askaripoor et al. characterized centralized platforms integrating safety-critical and non-safety-critical software domains through hardware virtualization [3], enabling mixed-criticality applications on shared physical hardware [85].

The architectural shift involves layered abstractions. Patrick Uven et al. proposed modular architecture templates with formalized interfaces separating hardware platforms, middleware services, and application layers [33]. This structure supports dynamic reconfiguration and hierarchical resource management [33]. Lucas Mauser et al. detailed five-layer architectures comprising hardware, hardware-near software, operating system, middleware, and application software layers [2].

Zonal Architectures

Zonal architectures represent an evolution beyond domain-based centralization, organizing vehicle systems by physical location rather than functional domain. Studies identified Zone Control Units (ZCUs) managing regional clusters of sensors and actuators [1, 17]. This topology reduces wiring complexity by localizing communication within zones before aggregating to central compute platforms [132].

Johannes Stümpfle et al. described zonal architectures grouping HPCs into vehicle zones including ADAS, Cockpit, Powertrain, Body and Comfort, and Connectivity [4]. The architecture incorporates abstraction layers encapsulating processor topologies through hypervisors [4]. J. Lim et al. proposed framework-based zonal design balancing communication load distribution against wiring length constraints [5], demonstrating quantifiable improvements in both metrics through column generation methods [5].

Khaoula Sghaier et al. characterized zonal ECUs connecting to central HPCs via high-bandwidth Automotive Ethernet [20], replacing traditional function-specific ECUs with zone-organized processing [20]. This approach facilitates integration of V2X communication and enables multicast message transmission in zonal topologies [47].

Service-Oriented Architectures

Service-oriented architectures (SOA) emerged as a dominant paradigm across the literature. Nemanja Kukulicic et al. conducted a systematic mapping identifying SOA as the primary architectural solution for managing automotive software complexity [6]. The architecture decomposes vehicle functionality into independent, reusable services communicating through standardized interfaces [6].

Studies documented three-layer SOA implementations: a Component/Sensor/Actuator Layer for hardware interfaces, a Control/Middleware layer managing service interactions, and a Management/Application layer coordinating higher-level functions [6]. Kevin Klein et al. presented the CARISMA service mesh architecture separating control plane configuration from data plane communication across distributed HPCs [39]. This separation enables dynamic service deployment while maintaining runtime adaptability [39].

The transition from signal-based to service-oriented communication represents a fundamental paradigm shift. Tanuku SaiLakshmi Madhuri et al. characterized this evolution as moving from static signal wiring to dynamic, event-driven service invocation [7]. Sreeraj Arole et al. documented migration from monolithic Classic AUTOSAR to service-oriented Adaptive AUTOSAR, achieving 25% improvement in response time despite 30% higher resource consumption [8].

Distributed and Hybrid Architectures

Several studies explored distributed architectures retaining multiple processing nodes. Subhojeet Mukherjee et al. developed software-defined truck testbeds virtualizing ECU networks across distributed platforms [35], achieving sub-millisecond latency for inter-node communication [35]. Alex Richardson et al. implemented distributed telemetry systems using LTE connectivity, demonstrating 99.74% reliability for Vehicle-to-Cloud communication [113].

Hybrid architectures combining static and dynamic elements were also documented. A. Vetter et al. described hybrid designs integrating traditional signal-oriented components with service-oriented subsystems [28], enabling gradual migration without complete architectural overhaul [28]. This approach allows leveraging SOA benefits while preserving legacy functionality [28].

Friedrich Wattenberg et al. proposed system-of-systems (SoS) architectures treating vehicles as integrated constituents in broader mobility ecosystems [46]. This perspective enables distributed control flows spanning vehicle-internal systems, cloud services, and infrastructure components [46].

Development Frameworks and Standards

The literature identified comprehensive frameworks and standards enabling SDV development, centered primarily on AUTOSAR platforms with emerging alternatives.

AUTOSAR Classic and Adaptive Platforms

AUTOSAR emerged as the dominant standardization framework across studies. V. Bandur et al. detailed AUTOSAR Classic's layered structure with Application Layer, Runtime Environment (RTE), and Basic Software (BSW) modules [1]. The platform provides standardized interfaces between software components and hardware [1], enabling reusability across multiple vehicle platforms [37].

AUTOSAR Adaptive represents the evolution toward service-oriented, high-performance computing environments. Tanuku SaiLakshmi Madhuri et al. described Adaptive Platform Foundation and Services supporting dynamic service discovery and event-driven communication [9]. The platform targets applications requiring complex data processing, real-time updates, and flexible deployment [110].

Multiple studies documented specific AUTOSAR implementations. X. Yang et al. developed fault diagnosis systems using AUTOSAR version 3.1, achieving 98.70% fault detection success rates through standardized Diagnostic Communication Manager and Function Inhibition Manager components [126]. Sreeraj Arole et al. validated Adaptive AUTOSAR integration with battery charging systems, demonstrating compatibility with Vector and Elektrobit commercial stacks [8].

The transition between AUTOSAR variants poses integration challenges. Youssef Elkharaz et al. compared Classic and Adaptive platforms, noting Classic's suitability for safety-critical, resource-constrained applications versus Adaptive's support for complex data processing [110]. Studies emphasized Common Foundation frameworks enabling interoperability between both variants [1].

SOME/IP and Communication Middleware

Scalable service-Oriented MiddlewarE over IP (SOME/IP) emerged as the primary service communication protocol. Thomas Schulik et al. validated SOME/IP integration across AUTOSAR Adaptive and ROS 2 platforms, achieving 50% lower median CPU load on Adaptive AUTOSAR compared to baseline implementations [11]. The middleware enables standardized service-oriented communication while abstracting underlying hardware [11].

Namcheol Lee et al. addressed bidirectional inter-domain communication using CommonAPI version 3.2.4 with vsomeip middleware [38]. Their automated gateway implementation achieved 8.67ms round-trip time for ADAS-to-IVI communication and 8.89ms for reverse direction [38]. However, SOME/IP emerged as a performance bottleneck in certain scenarios [38].

Data Distribution Service (DDS) provided an alternative middleware approach. Zhenpeng Li et al. compared SOME/IP and DDS protocols, demonstrating DDS achieved 3.9 times shorter average communication times for TCP and 6 times shorter for UDP [12]. DDS supports simultaneous data delivery to multiple subscribers through multicast transmission, reducing network load [12]. L'or'ant Meszl'enyi et al. implemented DDS-based systems with layered timing isolation achieving consistent latencies under heavy interference [42].

Alternative Frameworks and Platforms

Beyond AUTOSAR, several alternative frameworks emerged. Johannes Stümpfle et al. evaluated Android Automotive Operating System (AAOS) featuring Hardware Abstraction Layer (HAL) decoupling software from hardware [27]. The platform provides extensible modular architecture but faces real-time performance constraints and dependency on Google ecosystem [27].

Robot Operating System (ROS 2) appeared as a prototyping framework. Thomas Schulik et al. demonstrated cross-platform deployment between AUTOSAR Adaptive and ROS 2, auto-generating 95% of function adapter code [11].

However, ROS 2 exhibited higher CPU utilization compared to AUTOSAR implementations [11].

Luca Cuomo et al. developed mixed-criticality platforms using RISC-V processors for AUTOSAR OS alongside Linux for advanced functions [83]. Integration employed DDS-XRCE middleware for inter-OS communication and PREEMPT RT for real-time performance, reducing worst-case latency by 99% [83].

Development Methodologies and Tools

Model-Based Design (MBD) appeared frequently as a development approach. Lei Liao et al. presented Vehicle Domain-Specific Language (VDSL) enabling graphical specification with automatic code generation [40]. VDSL demonstrated 71% reduction in development effort and 75% cost reduction across three vehicle models at GAC Group [40].

Jun Cao et al. proposed the RoHo-ASI framework tailoring agile methodologies to automotive constraints [34]. The framework integrates role-oriented collaboration with hardware-in-the-loop testing and dynamic sprint cycles aligned with ISO 26262 standards [34]. Industrial validation showed earlier issue identification and higher resolution rates compared to traditional agile approaches [34].

Krzysztof Lebioda et al. introduced AI-powered workflows using Large Language Models for requirement processing, formal model generation, and deployment specification creation [69]. This approach automates significant portions of the development pipeline while requiring human oversight for validation [69].

F. Pan et al. developed automated Design Space Exploration (DSE) tools using Model-Based Systems Engineering (MBSE) with Eclipse Modeling Framework [65]. The approach transforms user specifications into solver-independent optimization formulas processed by Z3 SMT solver [65], enabling automated resource allocation decisions [65].

Standards Compliance and Safety Frameworks

Safety standards emerged as critical constraints. Multiple studies referenced ISO 26262 compliance for functional safety [1, 85, 108]. Ousman Khan et al. developed heuristic allocation strategies explicitly addressing ISO 26262 requirements for mixed-criticality application deployment [45]. The approach maintained strict constraint satisfaction while scaling to 800 applications [45].

Security standards received increasing attention. Khaoula Sghaier et al. discussed ISO/SAE 21434 and ISO/SAE 24089 compliance for cybersecurity and OTA updates [20]. Jaewan Seo et al. developed formally verified Software Update Management Systems using Event-B to demonstrate compliance with UN Regulation 156 [103]. Kim Strandberg et al. proposed UniSUF framework addressing security requirements across multiple update scenarios from OTA to factory deployment [91].

Jacqueline Henle et al. integrated OTA update requirements from ISO 24089 and UN R156 into traditional V-model development processes [134]. The unified process model addresses compliance with overlapping standards including ASPICE, ISO 26262, and ISO 21434 [134].

Technological Advancements Enabling Software-Defined Vehicles

The literature documented extensive technological innovations across computing platforms, communication systems, software update mechanisms, and integration approaches.

Computing Platform Innovations

High-Performance Computing platforms represent the foundational advancement enabling SDVs. Studies characterized HPCs as centralized processing units consolidating previously distributed ECU functions [3, 25]. V. Bandur et al. noted transition to multi-core CPUs providing substantially increased processing capacity compared to traditional microcontrollers [1].

Heterogeneous computing architectures combining general-purpose processors with specialized accelerators emerged frequently. H. Askaripoor et al. described HPCUs integrating multi-core SoCs, GPUs, RAM, and deep learning accelerators [3]. These platforms support AI-intensive tasks including object detection, sensor fusion, and autonomous driving functions [108].

Virtualization and containerization technologies enable resource partitioning on shared hardware. Anam Farrukh et al. implemented PC-class suspend-to-RAM techniques achieving 500ms resume time for critical CAN traffic processing [36]. The approach employs hardware virtualization partitioning resources between RTOS and Linux domains while maintaining timing guarantees [36].

Container orchestration appeared in multiple contexts. Anwar Ghammam et al. developed Docker-based container management achieving 35% energy reduction through optimized scheduling [15]. The system dynamically distributes workload across ECU clusters using NSGA-III multi-objective optimization [15], balancing energy, CPU, and memory consumption constraints [15].

Seong Hyeon Lim et al. proposed Zero Trust Hypervisor Architecture separating control plane verification from data plane transmission [50]. The architecture achieved improved round-trip time, throughput, and jitter while maintaining stability under attack loads [50].

Communication Technologies

Time-Sensitive Networking (TSN) emerged as the primary deterministic communication solution. Binqi Li et al. implemented Software-Defined TSN (SD-TSN) architecture automating scheduling computation and configuration [13]. The system achieved bounded latency stabilizing around 500 microseconds with minimal fluctuation [13], outperforming strict priority scheduling under varying interference loads [13].

Xu Zhong et al. developed automatic configuration for multicast routing and IEEE 802.1CB frame replication [44]. The system ensures message transmission reliability during link failures while reducing bandwidth waste through optimized multicast routing [44]. Integration with IEEE 802.1Qcc centralized configuration enables software-defined network management [44].

Software-Defined Networking (SDN) provides programmable network control. T. Häckel et al. integrated TSN with SDN, demonstrating Time-Sensitive Software-Defined Networking (TSSDN) enabling reconfiguration of synchronous traffic without delay penalties [55]. The architecture isolates control flows through OpenFlow-based differentiation, reducing attack surfaces while maintaining real-time performance [55].

Marco Häberle et al. proposed SDN architectures for automotive Ethernet accommodating high-bandwidth multimedia alongside time-critical control data [54]. The centralized controller manages network-wide configuration while local switches implement traffic shaping [54]. However, integration complexity and need for domain-specific protocols remain challenges [54].

Automotive Ethernet emerged as the backbone technology replacing legacy bus systems. V. Bandur et al. noted Ethernet adoption driven by bandwidth requirements exceeding CAN capabilities [86]. The technology supports gigabit data transfer rates with Time-Sensitive Networking ensuring deterministic latency [49].

Vehicle-to-Everything (V2X) communication technologies appeared in connected vehicle contexts. Yichen Luo et al. evaluated Cellular V2X (C-V2X) for OTA updates, demonstrating over 30 times faster transmission than Wi-Fi and up to 2,339 times faster using 5G versus V2X [106]. C-V2X enables updates even while vehicles remain in motion [106].

Software Update Mechanisms

Over-the-Air (OTA) update capabilities emerged as a defining SDV characteristic across nearly all studies. The technology enables continuous software evolution post-production, addressing vulnerabilities and adding features without physical service center visits [32, 40].

Merkle tree-based approaches provided security frameworks for updates. Abir Bazzi et al. developed MT-SOTA scheme achieving 80% timing reduction for signature verification [60]. The approach enables independent cluster updates while maintaining cryptographic integrity [60]. Extensions to product line engineering demonstrated 32-byte data savings per subtree with verification time reductions of 31-74 microseconds [84].

Ali Shoker et al. proposed ScalOTA architecture using update stations at EV charging locations [14]. This edge-based approach reduced bandwidth utilization and download latency by up to an order of magnitude compared to direct cellular repository-to-vehicle transmission [14]. The system provides formalized end-to-end chain-of-trust across manufacturers, suppliers, update stations, and ECUs [14].

Incremental update mechanisms emerged for efficiency. Yichen Luo et al. demonstrated incremental updates reducing transmission delays by 2 times versus full updates [106]. Using 32KB chunk sizes with C-V2X achieved up to 5,000 times transmission reduction compared to full package downloads [106]. Docker Compose orchestration enabled rolling updates minimizing service disruptions [106].

Arpan Bhattacharjee et al. developed edge-assisted frameworks evaluating update performance across varying distances and model sizes [32]. The approach addresses heterogeneity in software models and computing units while considering communication constraints [32]. Results showed update latency increasing with distance and model size, with bandwidth decreasing proportionally [32].

F. Pan et al. implemented automated platform configuration shifting integration from development to runtime [78]. The system combines Model-Based Systems Engineering with virtualization technologies, generating deployment plans dynamically through REST API communication [78]. Integration manager components orchestrate deployment using Terraform and Kubernetes extensions [78].

Virtualization and Containerization Technologies

Hypervisor-based virtualization enables mixed-criticality application isolation. F. Pan et al. employed hypervisor technology deploying applications with different safety criticality levels onto independent virtual machines [85]. This approach ensures freedom from interference as required by ISO 26262 [85].

Multiple studies utilized seL4 microkernel for safety-critical virtualization. Michael A. Doran et al. deployed virtualized guests and containers on ARMv8 platforms [21], combining hypervisor isolation with TSN logical segmentation to enhance system assurance [21]. The architecture supports military vehicle requirements including deterministic communication and enhanced security [21].

Container technologies appeared frequently for application packaging. Marc Schindewolf et al. proposed dynamic orchestration of containerized services across distributed platforms [82]. The orchestrator tracks parameter changes

and manages deployment decisions based on resource requirements and service dependencies [82]. Priority metrics describe service impact and propagate through dependency chains [82].

Ronak Indrasinh Kosamia et al. developed microkernel-inspired frameworks for Android-based IVI systems [123]. The modular architecture achieved 120-250 millisecond service recovery times while maintaining domain isolation [123]. Dynamic module loading leverages Android's service-oriented architecture and IPC mechanisms [123].

Zixuan Guo et al. reviewed virtualization techniques comprehensively, noting SR-IOV reduces computational overhead to 1.76% and latency by 40% for small messages [17]. Paravirtualization exhibits 4% performance overhead but requires better hardware compatibility [17]. Virtual CAN controllers enable testing without physical hardware while introducing potential latency concerns [17].

AI and Machine Learning Integration

AI integration patterns varied across functional domains. Md. Nuruzzaman et al. documented comprehensive AI deployment including convolutional neural networks, support vector machines, and unsupervised learning for fault classification [24]. These techniques enable predictive maintenance shifting from reactive to proactive strategies [24].

Arpan Bhattacharjee et al. evaluated machine learning model updates including ResNet-18 (46.8 MB), ResNet-50 (102.5 MB), and Faster R-CNN (175.2 MB) across varying transmission distances [32]. The study quantified OTA performance for AI model deployment in edge-assisted architectures [32].

Chaeyeong Lee et al. noted Baidu's Apollo platform incorporating learning-based vehicle dynamic modeling for autonomous driving [10]. However, many studies discussed AI requirements without specifying implementation details [51, 76, 87].

Krzysztof Lebioda et al. proposed Large Language Model integration for automated requirement processing, model generation, and deployment specification [69]. The approach demonstrates AI-assisted development workflows though requiring human supervision for validation [69]. Denesa Zyberaj et al. similarly employed LLMs and Vision-Language Models for test artifact generation, achieving 89% transformation rate from requirements to executable scenarios [48].

Digital twin technologies appeared in diagnostic and development contexts. Elisa Calisesi et al. proposed federated digital twin architectures enabling real-time vehicle monitoring and iterative validation [117]. Krish Agrawal et al. described digital twin-based attestation for OTA updates, simulating compatibility testing in virtual environments before physical vehicle deployment [101].

Networking and SDN Technologies

Software-Defined Networking extends beyond traditional automotive domains. N. Ferreira et al. implemented SDN through Open vSwitch in On-Board Units and Road Side Units [88], achieving sub-20ms handover duration with negligible overhead [88]. The system manages multihoming across heterogeneous wireless technologies including ITS-G5, 5G, and Ethernet [88].

Wanke Cao et al. applied SDN to driveline control in electric vehicles, implementing centralized frameworks with strategy, control, and data planes [16]. Hardware-in-loop testing demonstrated 87.5% reduction in loop delays through fractional-type basic period scheduling [16]. The hierarchical control scheme employs delay-embedded controllers and H-infinity LQR for stability against information delays [16].

Named Data Networking (NDN) emerged as an alternative paradigm. Zachariah Threet et al. demonstrated NDN providing secure bridges between CAN and Ethernet link layers [100]. Digital signing of data packets prevents unauthorized ECU data injection while supporting DoS and replay attack mitigation [100]. Pre-generating signed packets reduced response time by a factor of 2 [100].

Security and Safety Technologies

Security architectures evolved substantially. Seong Hyeon Lim et al. developed Zero Trust Hypervisor Architecture separating control plane session verification from data plane acceleration [50]. Gateway VMs enforce multi-layered security policies including VM identity and service access rights before establishing fast paths in Open vSwitch [50].

Khaoula Sghaier et al. documented multi-layered security measures including Software Bill of Materials (SBOM) for configuration transparency, Uptane and in-toto for supply chain security, and Hardware Security Modules (HSM) with Trusted Execution Environments (TEE) for hardware-level protection [97]. These technologies address expanded attack surfaces in connected vehicle environments [97].

Christian Prehofer et al. explored Self-Sovereign Identity (SSI) frameworks using decentralized identifiers and verifiable credentials [102]. SSI enables flexible trust relationships for components, services, and stakeholders while supporting traceability and authentication requirements [102]. Integration with blockchain provides distributed trust infrastructure [102].

Guilherme A. Thomaz et al. combined Intel SGX at servers with ARM TrustZone at clients for OTA update protection [135]. Experiments comparing CACIC-DevKit and Gramine-SGX implementations showed Gramine-SGX doubled latency despite development simplification [135], suggesting CACIC-DevKit better suits high-mobility scenarios [135].

eBPF/XDP technologies optimize kernel-level packet processing. SeongHyeon Lim et al. offloaded SOME/IP service discovery to kernel space, reducing context switching and CPU resource consumption [43]. Early packet filtering and efficient data sharing via eBPF maps improved overall performance [43]. Similarly, L'or'ant Meszl'enyi et al. used express data path (XDP) routing high-priority data directly from network interface drivers into critical application memory [42].

Hardware-Software Integration Technologies

Hardware abstraction layers emerged as critical decoupling mechanisms. Akshay Narla et al. evaluated HAL concepts including middleware-based approaches offering portability and modularity versus hypervisor-based solutions supporting safety and hardware efficiency [22]. Recommended hybrid designs integrate hypervisor isolation with middleware standardization [22].

Michael Paulweber et al. emphasized unified software interfaces and standardized hardware abstraction for sensors, actuators, computing, and storage [25]. These abstractions reduce supplier lock-in while maintaining compatibility across diverse hardware configurations [25]. The HAL4SDV project specifically targets development of technologies and processes for SDV advancement beyond 2030 [25].

Patrick Uven et al. proposed modular architecture templates with formalized resource requirement specifications [33]. The templates support runtime monitoring and systematic integration testing while enabling dynamic reconfiguration [33]. Update-compatibility concepts and Resource-and-Metadata definitions facilitate continuous validation [33].

Asset Administration Shell (AAS) frameworks appeared as interoperability solutions. Akshay Narla et al. investigated AAS potential for abstraction, modularity, and cross-platform data integration [124]. Applications include sensor abstraction and distributed testing scenarios [124], though implementation complexity and real-time constraints present challenges [124].

Effects of SDV Architectures on System Performance

Performance Metrics Across Architectural Approaches

Quantitative performance analyses revealed substantial improvements over traditional approaches, though with notable trade-offs. The centralized versus distributed architecture debate manifests through measurable differences in latency, resource utilization, and system efficiency.

Communication latency measurements demonstrated mixed results depending on architectural choices. Namcheol Lee et al. measured 8.67ms round-trip time for ADAS-to-IVI bidirectional communication [38], representing acceptable performance for non-safety-critical integration but identifying SOME/IP as a bottleneck requiring optimization [38]. Binqi Li et al. achieved 500 microsecond latency distribution with minimal fluctuation using SD-TSN mechanisms [13], demonstrating deterministic performance critical for safety applications.

Resource utilization efficiency varied significantly across middleware implementations. Thomas Schulik et al. reported 50% median CPU load for AUTOSAR Adaptive compared to 100% for ROS 2 on identical hardware [11], with 95% of function adapter code auto-generated reducing configuration time to under one minute [11]. Conversely, Sreeraj Arole et al. documented SOA architectures achieving 25% response time improvement but requiring 30% higher CPU and memory utilization compared to Classic AUTOSAR [8].

Communication protocol comparisons revealed substantial performance differences. Zhenpeng Li et al. measured DDS achieving 3.9 times shorter communication times versus SOME/IP for TCP and 6 times shorter for UDP [12]. DDS multicast transmission reduced network load through simultaneous delivery to multiple subscribers [12]. This advantage positions DDS favorably for high-throughput, multi-subscriber scenarios common in sensor fusion applications.

System startup and resume latency emerged as critical metrics. Anam Farrukh et al. demonstrated JuMP2start achieving 500ms resume time for critical CAN traffic with steady-state throughput reached in under 7ms [36]. This represents 52.5% improvement over traditional cold-start systems [36], enabling time-aware stop-start functionality essential for electric vehicle efficiency.

Scalability and Computational Efficiency

Optimization approaches demonstrated impressive scalability characteristics. F. Pan et al. evaluated Integer Linear Programming solvers showing exponential solving time increases with application numbers, with Gurobi achieving shortest solution times [85]. However, Ousman Khan et al. demonstrated heuristic strategies completing even 800-application problems in sub-second time [45], representing two to three orders of magnitude runtime reduction compared to exact solvers while requiring only modest VM usage increases [45].

Container orchestration efficiency varied by workload characteristics. Anwar Ghammam et al. achieved 35% energy consumption reduction in power-saving mode compared to Ford Motor Company's baseline scheduler [15]. Migration times ranged from 156ms to 30ms with negligible overhead [15], enabling dynamic workload distribution maintaining high-priority container availability during failures [15].

Development efficiency improvements appeared across multiple frameworks. Lei Liao et al. reported VDSL achieving 71% development effort reduction and 75% cost reduction across three GAC Group vehicle models over 18 months [40]. The low-code approach streamlined subscription handling and signal processing while enabling flexible OTA updates [40].

Safety and Real-Time Performance

Timing isolation emerged as critical for mixed-criticality systems. L'or'ant Meszl'enyi et al. demonstrated layered architectures achieving consistent and predictable latencies for real-time traffic under heavy best-effort interference [42]. The approach integrates fixed-priority middleware scheduling, express data path network routing, and dedicated NIC queues [42].

Virtualization performance overhead required careful management. Zixuan Guo et al. documented SR-IOV reducing computational overhead to 1.76% with 40% latency reduction for small messages [17]. Paravirtualization exhibited 4% overhead but demands enhanced hardware support [17]. These measurements inform design decisions balancing safety isolation against performance requirements.

Diagnostic system performance showed significant variation. X. Yang et al. achieved 98.70% fault detection success with 40.68% single-core CPU utilization and 0.0217 second detection time [126]. This represents 12.97% improvement over CAN bus methods and 8.98% over structural analysis approaches [126], demonstrating enhanced diagnostic accuracy through AUTOSAR-based architectures.

Network configuration automation enhanced determinism. Binqi Li et al. demonstrated SD-TSN outperforming strict priority scheduling by maintaining consistent latency and jitter under varying interference loads [13]. Automated configuration reduced developer workload while eliminating manual configuration errors [13].

Security Performance and Attack Mitigation

Security architectures demonstrated measurable resilience improvements. Seong Hyeon Lim et al. showed Zero Trust Hypervisor Architecture maintaining critical system stability under attack loads while improving RTT, throughput, and jitter [50]. The verify-first, then-accelerate approach implements Zero Trust principles with minimal performance degradation [50].

Update security mechanisms balanced protection against efficiency. Abir Bazzi et al. reported MT-SOTA reducing verification times from milliseconds to microseconds through optimized Merkle tree design [60]. However, efficiency diminishes with fewer software clusters requiring careful tree structure design [60]. Yunje Shin et al. demonstrated MQTree achieving approximately four times processing time improvement over HTTPS alone [98] through MQTT with TLS and Merkle tree blockchain verification [98].

Control flow isolation reduced attack surfaces. T. Häckel et al. measured TSSDN achieving lower end-to-end latency than pure TSN while enabling dynamic reconfiguration without penalties [55]. Exposed embedding of control flow identifiers enabled network infrastructure to establish trust zones, largely reducing connected car attack surfaces [89] while minimizing communication expenses [89].

Cost and Development Efficiency

Wiring harness reductions provided tangible cost benefits. V. Bandur et al. reported 30% savings in wiring mass and cost from Delphi implementations, with Bosch achieving 15-20% weight reductions [1]. Zixuan Guo et al. documented centralized E/E architectures reducing wiring harness by 20% while scaling down ECU counts by 20% [17].

Development timeline improvements emerged through multiple mechanisms. Jun Cao et al. reported RoHo-ASI framework achieving earlier issue identification and higher resolution rates than traditional agile approaches [34]. The framework aligns rapid iteration with ISO 26262 requirements through embedded traceability and modular safety cases [34].

Software reuse benefits appeared consistently. Ravinder Katla et al. noted AUTOSAR significantly reducing development costs and timeframes while maintaining consistency across vehicle lines [37]. The modular design principles support flexible architectures accommodating evolving technological demands [37].

Synthesis: Architectural Trade-offs and Context Dependencies

The literature reveals that architectural performance outcomes depend critically on specific deployment contexts, application requirements, and technology maturity levels rather than representing universal superiorities.

Centralized Versus Distributed Architecture Performance

Studies reporting superior centralized architecture performance primarily focused on wiring cost reduction, software reuse efficiency, and update management simplicity. V. Bandur et al. documented 15-30% wiring harness reductions in domain-centralized implementations [1], while centralized platforms enabled consolidated OTA update distribution [1]. These benefits manifest most strongly in scenarios prioritizing physical integration efficiency and unified software management.

Conversely, distributed architectures demonstrated advantages in fault isolation and system resilience. Marc Schindewolf et al. emphasized distributed orchestrator approaches avoiding single points of failure [82], particularly critical for safety-relevant applications. Subhojeet Mukherjee et al. achieved sub-millisecond latency through distributed network virtualization [35], superior to centralized processing latencies in bandwidth-constrained scenarios.

The apparent contradiction resolves through workload characterization. Centralized architectures excel for compute-intensive tasks requiring GPU acceleration or AI processing, as demonstrated by H. Askaripoor et al.'s integration of deep learning accelerators in HPCUs [3]. Distributed approaches remain optimal for latency-critical control loops and safety-isolated subsystems where communication overhead to central processors introduces unacceptable delays.

Service-Oriented Architecture Performance Context

SOA implementations showed divergent resource utilization patterns depending on middleware selection. Sreeraj Arole et al.'s 30% higher CPU and memory usage in SOA versus monolithic Classic AUTOSAR [8] contrasts with Thomas Schulik et al.'s 50% lower CPU load for Adaptive AUTOSAR [11]. This discrepancy reflects middleware maturity and optimization levels rather than fundamental SOA limitations.

The timing context matters substantially. Studies measuring response time improvements (25% reduction in Sreeraj Arole et al. [8]) focused on application-level latencies, while investigations reporting higher resource consumption examined underlying middleware overhead. Both findings remain valid within their measurement contexts—SOA improves application responsiveness through asynchronous communication patterns while requiring additional processing for service discovery and message routing.

Implementation scale influences outcomes significantly. Lei Liao et al.'s 71% development effort reduction with VDSL [40] applied across production deployment spanning three vehicle models over 18 months, representing mature toolchain integration. Proof-of-concept studies naturally show different efficiency characteristics reflecting

prototype overhead absent in production systems.

OTA Update Performance Dependencies

Update mechanism performance exhibits strong dependency on network topology and file size characteristics. Yichen Luo et al.'s demonstration of C-V2X achieving 30 times faster transmission than Wi-Fi and 2,339 times faster than V2X using 5G [106] applies specifically to large model downloads (ResNet, YOLO weights) where high bandwidth dominates performance. Ali Shoker et al.'s order-of-magnitude improvements using edge update stations [14] similarly assume large software packages benefiting from reduced cellular transmission.

For small, incremental updates, security overhead becomes proportionally more significant. Abir Bazzi et al.'s 80% timing reduction through MT-SOTA [60] demonstrates security optimization impact when verification overhead dominates total update time. However, efficiency diminishes with fewer software clusters [60], indicating applicability primarily for highly modular architectures with frequent independent component updates.

Network topology critically influences optimal strategies. Yichen Luo et al. showed 32KB chunk sizes achieving highest success rates across distances [106], while Ali Shoker et al.'s wired update station approach avoided wireless variability entirely [14]. Both approaches prove effective within their deployment scenarios—moving vehicles benefit from cellular resilience despite higher latency, while stationary charging scenarios leverage deterministic wired connections.

Virtualization Trade-offs and Application Domains

Virtualization performance overhead varies substantially by approach and application criticality. Zixuan Guo et al.'s finding that SR-IOV reduces overhead to 1.76% while paravirtualization exhibits 4% overhead [17] applies primarily to I/O-intensive applications where direct hardware access provides measurable benefits. Safety-critical applications may accept higher overhead for enhanced isolation guarantees.

The 99% worst-case latency reduction achieved by Luca Cuomo et al. using PREEMPT RT [83] demonstrates that software optimizations sometimes exceed hardware virtualization benefits. This suggests mixed-criticality architectures should combine virtualization for spatial isolation with OS-level optimizations for temporal guarantees rather than relying exclusively on either approach.

Container versus virtual machine trade-offs manifest through different performance profiles. Ronak Indrasinh Kosamia et al.'s 120-250ms service recovery times using Android containers [123] provide faster restart than full VM initialization but offer weaker isolation boundaries. The choice depends on fault recovery requirements versus isolation strength needs for specific application domains.

Middleware Selection Performance Implications

Communication middleware selection significantly impacts system-level performance beyond simple protocol efficiency. The 6 times shorter DDS communication time versus SOME/IP for UDP reported by Zhenpeng Li et al. [12] reflects fundamental architectural differences—DDS's publisher-subscriber model with multicast transmission suits many-to-many data flows, while SOME/IP's request-response patterns optimize point-to-point service invocation.

The 50% CPU load advantage for AUTOSAR Adaptive versus ROS 2 reported by Thomas Schulik et al. [11] does not indicate universal AUTOSAR superiority. Rather, it demonstrates production-grade middleware optimization versus research-oriented platform overhead. ROS 2's flexibility advantages manifest in rapid prototyping contexts where development speed outweighs runtime efficiency.

Context switching optimization through kernel-level processing, as demonstrated by SeongHyeon Lim et al.'s eBPF/XDP implementation [43], provides benefits proportional to service discovery frequency. High-churn environments with frequent service topology changes gain substantially, while stable topologies with infrequent discovery amortize initial overhead differently.

Technical Challenges and Research Gaps

The literature identified substantial technical barriers requiring resolution for SDV technology maturation, spanning implementation complexity, safety assurance, standardization, and regulatory compliance.

Implementation Complexity and Integration Challenges

Software complexity emerged as the primary technical barrier. Alaa Khamis et al. identified rising complexity in software maintenance and repair as critical obstacles [19], compounded by fragmented certification frameworks and lack of standardized toolchains [19]. The evolution toward expectations of 1 billion lines of code for fully autonomous vehicles [18] exceeds current architectural capacity for comprehension and validation.

Hardware-software integration difficulties appeared across multiple dimensions. Thomas Schulik et al. noted cumbersome integration due to system-specific interface mappings [11], with limited hardware component reusability and non-standardized vehicle signals hampering cross-platform development [11]. Christian Hackenbeck et al. identified gaps in service interface interactions with internal architecture and external non-SOA components [66], hindering seamless integration despite SOA adoption.

Multi-domain communication complexity presents ongoing challenges. Dusan Kenjic et al. emphasized difficulties in establishing inter-domain connectivity requiring suitable protocol determination, specialized domain knowledge, and accommodation of software platform heterogeneity [99]. Namcheol Lee et al. addressed protocol compatibility, IDL translation, and language binding complexities in ADAS-IVI integration [38], challenges extending across all cross-domain scenarios.

Wanke Cao et al. documented unknown cross-domain multihop information delays and system uncertainties in domain-centralized architectures [16]. These unpredictable latencies degrade performance and potentially compromise stability [16], requiring sophisticated scheduling strategies like fractional-type basic period methods to achieve acceptable control response [16].

Safety and Security Compliance Challenges

Functional safety compliance with ISO 26262 faces new complexities in software-defined contexts. H. Askaripoor et al. noted significant safety requirements remain unaddressed in current architectural synthesis studies [3], particularly concerning verification of centralized platform safety properties. The consolidation of functions increases failure impact scope, complicating fault containment [1].

Safety certification processes struggle with dynamic software updates. Yuchu Liu et al. identified certification renewal requirements governed by legislation as barriers to rapid experimentation [120]. The tension between software update frequency and safety validation timescales creates fundamental process conflicts requiring new compliance frameworks.

Cybersecurity challenges expanded with increased connectivity. Khaoula Sghaier et al. documented broadened attack surfaces from software-centric architectures [20], requiring multi-layered defenses including SBOM transparency, supply chain security, and hardware-level protection [97]. Regulatory landscapes strain from convergence

of automotive, telecommunications, and software domains requiring unified guidelines [20].

Security integration with real-time performance poses technical conflicts. Khaoula Sghaier et al. identified challenges balancing real-time safety-critical functionalities with real-time attack detection and response [20]. Memory and processing limitations in legacy ECUs complicate security measure deployment [20], while scalability of security infrastructure becomes critical as SDV numbers grow [20].

Jaewan Seo et al. noted incompatibility between formal verification tools and practical implementation [103], with generated source code requiring substantial refinement for actual system compilation [103]. This gap between theoretical safety assurance and practical deployment hampers certification processes.

Standardization Gaps and Interoperability Issues

Interface standardization remains critically incomplete. Christian Hackenbeck et al. identified static AUTOSAR interface definitions limiting adaptability [66], while non-standardized SOA approaches offer flexibility at the cost of interoperability [66]. Harmonized standards balancing flexibility and compatibility remain elusive.

Communication protocol heterogeneity creates integration barriers. Thomas Schulik et al. documented non-standardized vehicle signals arising from security and proprietary concerns [11], with variability in signal definitions among manufacturers [11]. This fragmentation necessitates manual interface mapping processes hampering cross-platform development efficiency [11].

Development tool compatibility gaps emerged frequently. Thomas Schulik et al. noted lack of compatibility across frameworks and vendors [11], requiring manual processes for adapter code generation. Christian Hackenbeck et al. emphasized needs for standardized interface patterns and hardware abstraction layers enabling service-component integration [66].

Diagnostic protocol evolution lags architectural advances. Boris Böhlen et al. identified ASAM SOVD lacking solutions for diagnosing service-oriented architecture aspects [23], including service quality validation, chain-of-effects analysis, and dynamic resource usage monitoring [23]. Conventional data center monitoring approaches prove inapplicable due to vehicle cost constraints [23].

Real-Time Performance and Determinism Challenges

Achieving deterministic communication over Ethernet-based systems presents fundamental challenges. Michael A. Doran et al. noted Ethernet's inherent lack of determinism as a significant disadvantage for safety-critical applications [21], requiring TSN enhancements to provide bounded message latency [21]. Configuration complexity of TSN mechanisms creates deployment barriers [54].

Linux-based platforms introduce timing unpredictability. L'or'ant Meszl'enyi et al. documented concurrent network stack usage creating unpredictable latency and jitter [42], necessitating multi-layer isolation strategies. The challenge intensifies for mixed-criticality scenarios requiring both best-effort and hard real-time traffic coexistence.

Virtualization real-time performance remains problematic. V. Bandur et al. identified unpredictable real-time performance in virtualization as a key limitation [1], with high memory usage and complexities in peripheral virtualization [1]. These constraints particularly affect safety-critical functions with stringent timing requirements.

Software update timing conflicts with operational requirements. Patrick Uven et al. noted reduced computation resource availability can force timing deadline misses [33], requiring sophisticated runtime monitoring and dynamic resource reallocation. Integration test execution in constrained automotive platforms proves challenging [33].

Scalability and Deployment Barriers

Resource allocation complexity scales poorly with application growth. F. Pan et al. documented exponential solving time increases as application numbers grow in Integer Linear Programming approaches [85], though heuristic methods mitigate this challenge for large-scale deployments [45]. The transition from manageable dozens of ECUs to hundreds of software services creates qualitatively different architectural challenges.

Update management scalability faces substantial barriers. Ali Shoker et al. identified storage scalability challenges from numerous vehicle model variants [14], while Abir Bazzi et al. noted coordination complexity among multiple software providers [84]. The combinatorial explosion of configuration variants across model lines and option packages strains traditional update approaches.

Testing and validation scalability lags software complexity growth. Johannes Stümpfle et al. emphasized insufficient traditional testing methods for SDV's rapid update cadence and complex function interplay [4]. Automated test generation using GenAI achieved only 89% transformation rate from requirements to executable scenarios [48], with human review remaining necessary [48].

Infrastructure dependencies create deployment barriers. Yichen Luo et al. noted varying automotive regulations across countries and states complicating standardized deployments [106], while network coverage issues affect OTA update reliability [106]. ScalOTA's dependency on charging station infrastructure limits applicability to electric vehicle contexts [14].

Future Research Directions

Multiple critical research gaps emerged across studies. Alaa Khamis et al. identified needs for ecosystem collaboration, open innovation, standardized datasets, education simulators, quantifiable Quality of Experience metrics, cognitive data sampling, and contextual observability [19]. These requirements span technical capabilities, industry coordination, and research infrastructure.

Hardware abstraction standardization remains incomplete. Akshay Narla et al. recommended hybrid HAL designs integrating hypervisor isolation with middleware standardization [22], though optimal integration patterns require further investigation. J. Lee et al. identified needs for advanced hardware abstraction supporting standardization, adaptive intelligence for transparency, unified data layers, and redefined product module architectures [68].

Security architecture evolution requires continued development. Khaoula Sghaier et al. called for proactive security approaches and advanced anomaly detection systems [20]. Christian Prehofer et al. identified needs for systematic comparison between SSI and existing standards like X.509 [102], along with reliable long-term storage for traceability [102].

Diagnostic and monitoring capabilities lag architectural evolution. Sandra Bickelhaupt et al. emphasized adapting cloud-based observability approaches for in-vehicle software systems [31], while Boris Böhlen et al. identified specific gaps in service quality validation, chain-of-effects analysis, and historical data analysis for SOA environments [23].

Skill gaps and organizational challenges emerged as non-technical barriers. Md. Nuruzzaman et al. documented widening skill gaps in diagnostics engineering particularly for AI and systems integration [24], hampering effective technology adoption. Johannes Stümpfle et al. noted ecosystem architecture redesign requirements and unclear open source software administration [4], affecting industry-wide coordination essential for SDV success.

Regulatory frameworks require substantial evolution. Khaoula Sghaier et al. identified regulatory landscapes strained by automotive-telecommunications-software convergence [20], while Johannes Stümpfle et al. noted out-

dated regulatory frameworks inadequate for current SDV capabilities [4]. Standards development must accelerate to match technological advancement pace.

References

1. V. Bandur, Gehan M. K. Selim, Vera Pantelic, M. Lawford (2021) Making the Case for Centralized Automotive E/E Architectures. IEEE Transactions on Vehicular Technology. https://doi.org/10.1109/TVT.2021.3054934

2. Lucas Mauser, Eva Zimmermann, Pavel Nedvedický, et al (2025) Towards Mixed-Criticality Software Architectures for Centralized HPC Platforms in Software-Defined Vehicles: A Systematic Literature Review. European Conference on Software Architecture. https://doi.org/10.48550/arXiv.2506.05822

3. H. Askaripoor, Morteza Hashemi Farzaneh, A. Knoll (2022) E/E Architecture Synthesis: Challenges and Technologies. Electronics. https://doi.org/10.3390/electronics11040518

4. Johannes Stümpfle, Johannes Sigel, Matthias Weiss, et al (2025) The Software-Defined Vehicle: A Comprehensive Study on Current Trends and Challenges. IEEE Engineering Management Review. https://doi.org/10.1109/emr.2025.3636574

5. J. Lim, Jongheon Lee, Y. S. Hong, Changmuk Kang (2025) A Framework for Designing Zonal Architectures for In-Vehicle Networks: Balancing Communication Load and Wiring Length. IEEE Transactions on Vehicular Technology. https://doi.org/10.1109/TVT.2024.3522659

6. Nemanja Kukulicic, Damjan Samardzic, Alessio Bucaioni, S. Mubeen (2022) Automotive Service-oriented Architectures: a Systematic Mapping Study. EUROMICRO Conference on Software Engineering and Advanced Applications. https://doi.org/10.1109/SEAA56994.2022.00077

7. Tanuku SaiLakshmi Madhuri, B. J., Prasad Patil (2025) A Model-Based Design Approach Towards Software-Defined Vehicles with Service-Oriented Architecture. 2025 3rd International Conference on Recent Advances in Information Technology for Sustainable Development (ICRAIS). https://doi.org/10.1109/ICRAIS66073.2025.11234710

8. Sreeraj Arole (2023) From Monolith to Service-Oriented Architecture: A Model-Based Design Approach towards Software-Defined Vehicles. 2023 5th International Conference on Electrical, Control and Instrumentation Engineering (ICECIE). https://doi.org/10.1109/ICECIE58751.2024.10457489

9. Tanuku SaiLakshmi Madhuri, J. B. Vishnu (2025) Software-Defined Vehicles: The Future of Automobile Industry. International Conference on Communication Systems and Networks. https://doi.org/10.1109/COMSNETS63942.2025.10885701

10. Chaeyeong Lee, Moonbeom Kim, Hyewon Seo, Jeongyeup Paek (2022) Survey on Integrated Vehicular Platforms for Next Generation Mobility. Information and Communication Technology Convergence. https://doi.org/10.1109/ICTC55196.2022.9952950

11. Thomas Schulik, Viswanatha Reddy Batchu, Ramesh Kumar Dharmapuri, et al (2025) Efficient Integration of cross platform functions onto service-oriented architectures. arXivorg. https://doi.org/10.48550/arXiv.2510.27344

12. Zhenpeng Li, Jingwei Bian, Wei Li, Lihua Zhang (2025) Safety measures in automotive operating system: a comprehensive review of trends and defense frameworks. Journal of Supercomputing. https://doi.org/10.1007/s11227-025-07635-7

13. Binqi Li, Yuan Zhu, Qin Liu, Xiangxi Yao (2024) Development of Deterministic Communication for In-Vehicle Networks Based on Software-Defined Time-Sensitive Networking. Machines. https://doi.org/10.3390/machines12110816

14. Ali Shoker, Fernando Alves, Paulo Esteves-Verissimo (2023) ScaIOTA: Scalable Secure Over-the-Air Software Updates for Vehicles. IEEE International Symposium on Reliable Distributed Systems. https://doi.org/10.1109/SRDS60354.2023.00024

15. Anwar Ghammam, Rania Khalsi, Marouane Kessentini, Foyzul Hassan (2024) Efficient Management of Containers for Software Defined Vehicles. ACM Transactions on Software Engineering and Methodology. https://doi.org/10.1145/3672461

16. Wanke Cao, Zhiwen Zhu, Jinrui Nan, et al (2024) Combined LKA and DYC Control for Electric Vehicle With a Domain-Centralized E/E Architecture Based on Software-Defined Networking. IEEE Transactions on Transportation Electrification. https://doi.org/10.1109/tte.2023.3320262

17. Zixuan Guo, K. Koufos, M. Dianati, Roger Woodman (2025) State-of-the-art virtualisation technologies for the centralised automotive E/E architecture. Frontiers in Future Transportation. https://doi.org/10.3389/ffutr.2025.1519390

18. Pedro Teixeira, Duarte M. G. Raposo, R. Lopes, Susana Sargento (2024) Deterministic and Reliable Software-Defined Vehicles: key building blocks, challenges, and vision

19. Alaa Khamis, Partha Goswami (2025) Rethinking Vehicle Architecture Through Softwarization and Servitization. IEEE Access. https://doi.org/10.1109/ACCESS.2025.3588432

20. Khaoula Sghaier, Badis Hammi, Ghada Gharbi, et al (2025) Advancing Security in Software-Defined Vehicles: A Comprehensive Survey and Taxonomy. arXivorg. https://doi.org/10.48550/arXiv.2510.09675

21. Michael A. Doran, Mark G. Russell, Leonard Elliott (2024) Deterministic & Modular Architecture for Embedded Vehicle Systems. SAE technical paper series. https://doi.org/10.4271/2024-01-4033

22. Akshay Narla, Johannes Stümpfle, Souvik Saha, et al (2025) Evaluating Hardware Abstraction Layer Concepts for Software Defined Vehicles: Insights into Applicability and Effectiveness. 2025 IEEE International Automated Vehicle Validation Conference (IAVVC). https://doi.org/10.1109/IAVVC61942.2025.11219503

23. Boris Böhlen, Diana Fischer (2026) Monitoring and Diagnosing Software-Defined Vehicle Architectures. SAE technical paper series. https://doi.org/10.4271/2026-26-0677

24. Md. Nuruzzaman (2025) AUTOMOTIVE SYSTEM RELIABILITY AND TECHNOLOGICAL CONVERGENCE: A REVIEW OF SMART POWERTRAIN AND MECHATRONIC DIAGNOSTICS. American Journal of Scholarly Research and Innovation. https://doi.org/10.63125/xgp15k85

25. Michael Paulweber, Andreas Eckel, Paolo Azzoni (2025) Multi-Partner Project: Shaping the Vehicle of the Future - How FEDERATE and HAL4SDV are Steering Europe's Software-Defined Vehicle Ecosystem. Design, Automation and Test in Europe. https://doi.org/10.23919/DATE64628.2025.10992971

26. David Fernández Blanco, Frédéric le Mouël, T. Lin, M. Escudie (2023) A Comprehensive Survey on Software as a Service (SaaS) Transformation for the Automotive Systems. IEEE Access. https://doi.org/10.1109/access.2023.3294256

27. Johannes Stümpfle, Akshay Narla, Nasser Jazdi, Michael Weyrich (2025) Android Automotive for Software-Defined Vehicles: An Assessment of Capabilities, Limitations, and Future Directions. 2025 IEEE International Automated Vehicle Validation Conference (IAVVC). https://doi.org/10.1109/IAVVC61942.2025.11219511

28. A. Vetter, Philipp Obergfell, Houssem Guissouma, et al (2020) Development Processes in Automotive Service-oriented Architectures. Mediterranean Conference on Embedded Computing. https://doi.org/10.1109/MECO49872.2020.9134175

29. Kyle Taylor (2024) Digital Cockpit in the Era of the Software-Defined Vehicle. SAE technical paper series. https://doi.org/10.4271/2024-01-2391

30. Brian L. Berg (2024) Service-Oriented Architecture Over UDSoCAN. 2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall). https://doi.org/10.1109/VTC2024-Fall63153.2024.10757791

31. Sandra Bickelhaupt, Michael Hahn, Andrey Morozov, Michael Weyrich (2024) Towards Future Vehicle Diagnostics in Software-Defined Vehicles. SAE technical paper series. https://doi.org/10.4271/2024-01-2981

32. Arpan Bhattacharjee, Hamza Mahmood, Sidi Lu, et al (2023) Edge-Assisted Over-the-Air Software Updates. International Conference on Communications in Computing. https://doi.org/10.1109/CIC58953.2023.00013

33. Patrick Uven, Ralf Stemmer, Gregor Nitsche (2025) A Modular Architecture Template for Resource Modeling in Software-Defined Vehicles. 2025 20th European Dependable Computing Conference Companion Proceedings (EDCC-C). https://doi.org/10.1109/EDCC-C66476.2025.00060

34. Jun Cao, Changming Zhu (2025) Re-imagining Automotive Software Development: RoHo-ASI Framework for SDV. 2025 IEEE 5th International Conference on Software Engineering and Artificial Intelligence (SEAI). https://doi.org/10.1109/SEAI65851.2025.11108774

35. Subhojeet Mukherjee, J. Daily (2021) Towards a Software Defined Truck. INCOSE International Symposium. https://doi.org/10.1002/j.2334-5837.2021.00884.x

36. Anam Farrukh, Richard West (2024) JuMP2start: Time-Aware Stop-Start Technology for a Software-Defined Vehicle System. Euromicro Conference on Real-Time Systems. https://doi.org/10.4230/LIPIcs.ECRTS.2024.1

37. Ravinder Katla (2025) AUTOSAR Framework: Enabling Scalable and Modular Software Architecture in Contemporary Automotive Systems. Journal of Information Systems Engineering & Management. https://doi.org/10.52783/jisem.v10i60s.13115

38. Namcheol Lee, Woobean Seo, Seongsoo Hong, Saehwa Kim (2026) A Protocol-Agnostic and Automated Approach to Bi-directional Inter-Domain Communication in Software-Defined Vehicles. International journal of automotive technology. https://doi.org/10.1007/s12239-025-00375-w

39. Kevin Klein, Pascal Hirmer, Steffen Becker (2024) CARISMA: CAR-Integrated Service Mesh Architecture. International Conference on Vehicle Technology and Intelligent Transport Systems. https://doi.org/10.5220/0012650300003702

40. Lei Liao, Junjie Wang, Zhen Xu, et al (2024) Vehicle Domain-Specific Language: Unifying Modeling and Code Generation for Low-Code Automotive Development. International Conference on Automated Software Engineering. https://doi.org/10.1145/3691620.3695282

41. Sandra Bickelhaupt, Michael Hahn, Michael Weyrich, Andrey Morozov (2025) An Architecture for Vehicle Diagnostics in Software-Defined Vehicles. SAE technical paper series. https://doi.org/10.4271/2025-01-0280

42. L'or'ant Meszl'enyi, Julius Kahle, Dominik Püllen, et al (2025) Towards Timing Isolation for Mixed-Criticality Communication in Software-Defined Vehicles. 2025 IEEE International Automated Vehicle Validation Conference (IAVVC). https://doi.org/10.1109/IAVVC61942.2025.11219644

43. SeongHyeon Lim, S. Oh, Syng Keun Cha, et al (2025) An eBPF/XDP-Based Architecture for Efficient SOME/IP Service Discovery. International Conference on Mechatronics and Robotics Engineering. https://doi.org/10.1109/ICMRE64970.2025.10976318

44. Xu Zhong, Yuan Zhu (2024) Configuration of Multicast Routing and IEEE 802.1CB in Time-Sensitive Networking for Software-Defined Vehicles. International Conference on Speech Technology and Human-Computer Dialogue. https://doi.org/10.1109/ICCT62411.2024.10946575

45. Ousman Khan, Alaa Khamis (2025) Heuristics-based Resource Allocation in Software-defined Vehicles. Global Conference on Artificial Intelligence and Internet of Things. https://doi.org/10.1109/GCAIoT68269.2025.11275567

46. Friedrich Wattenberg, Moritz Weigel, Till Fuchs, Bernard Bäker (2025) Functional Alternatives for Holistic Architectures in Software-Defined Vehicles. Conference on Computability in Europe. https://doi.org/10.1115/detc2025-163813

47. Sung Keun Cha, S. Oh, Y. Do, et al (2025) Service-Oriented Vehicle Diagnostics by Integrating DDS and DoIP. 2025 International Technical Conference on Circuits/Systems, Computers, and Communications (ITC-CSCC). https://doi.org/10.1109/ITC-CSCC66376.2025.11137625

48. Denesa Zyberaj, Lukasz Mazur, Pascal Hirmer, et al (2026) Req2Road: A GenAI Pipeline for SDV Test Artifact Generation and On-Vehicle Execution

49. Ganesh Pawar, Sumer Inamdar, Mayank Kumar, et al (2026) High-Performance Computing for Software-Defined EV Powertrains: Design and Real-Time Validation with NXP S32G3 Domain Controller. SAE technical paper series. https://doi.org/10.4271/2026-26-0684

50. Seong Hyeon Lim, S. Oh, Y. Do, et al (2025) ZTHA: A Zero Trust Hypervisor Architecture for Software-Defined Vehicle HPC. 2025 IEEE/IEIE International Conference on Consumer Electronics-Asia (ICCE-Asia). https://doi.org/10.1109/ICCE-Asia67487.2025.11263576

51. Aqueel Ahmad, K. Hemanth, Om Kumar, et al (2026) Software-Defined Vehicles: Architecting the Future of Intelligent and Connected Mobility. SAE technical paper series. https://doi.org/10.4271/2026-26-0691

52. Shu Jiang (2024) Vehicle E/E Architecture and Key Technologies Enabling Software-Defined Vehicle. SAE technical paper series. https://doi.org/10.4271/2024-01-2035

53. Haoxin Wang, Tingting Liu, Baekgyu Kim, et al (2020) Architectural Design Alternatives Based on Cloud/Edge/Fog Computing for Connected Vehicles. IEEE Communications Surveys and Tutorials. https://doi.org/10.1109/COMST.2020.3020854

54. Marco Häberle, Florian Heimgaertner, Hans Löhr, et al (2020) Softwarization of Automotive E/E Architectures: A Software-Defined Networking Approach. IEEE Vehicular Networking Conference. https://doi.org/10.1109/VNC51378.2020.9318389

55. T. Häckel, Philipp Meyer, Franz Korf, T. Schmidt (2022) Secure Time-Sensitive Software-Defined Networking in Vehicles. IEEE Transactions on Vehicular Technology. https://doi.org/10.1109/TVT.2022.3202368

56. Teresa Placho, Christoph Schmittner, Arndt Bonitz, Oliver Wana (2020) Management of automotive software updates. Microprocessors and microsystems. https://doi.org/10.1016/j.micpro.2020.103257

57. Lionel Nkenyereye, Lewis Nkenyereye, Bayu Adhi Tama, et al (2020) Software-Defined Vehicular Cloud Networks: Architecture, Applications and Virtual Machine Migration. Italian National Conference on Sensors. https://doi.org/10.3390/s20041092

58. Lionel Nkenyereye, Lewis Nkenyereye, Jong-Wook Jang (2023) Convergence of Software-Defined Vehicular Cloud and 5G Enabling Technologies: A Survey. Electronics. https://doi.org/10.3390/electronics12092066

59. V. N, Mitesh M Kumar, Balasubramanian Achuthan, et al (2023) Centralized E/E Architecture and Evolution. SAE technical paper series. https://doi.org/10.4271/2023-01-1511

60. Abir Bazzi, A. Shaout, Di Ma (2023) MT-SOTA: A Merkle-Tree-Based Approach for Secure Software Updates over the Air in Automotive Systems. Applied Sciences. https://doi.org/10.3390/app13169397

61. Sidi Lu, N. Ammar, Akila Ganlath, et al (2022) A Comparison of End-to-End Architectures for Connected Vehicles. 2022 Fifth International Conference on Connected and Autonomous Driving (MetroCAD). https://doi.org/10.1109/MetroCAD56305.2022.00015

62. S. Aust (2022) Vehicle Update Management in Software Defined Vehicles. IEEE Conference on Local Computer Networks. https://doi.org/10.1109/LCN53696.2022.9843360

63. M. J. Anjum, M. S. Farooq (2021) SDN based V2X Networks for Disaster Management: A Systematic Literature Review. VFAST Transactions on Software Engineering. https://doi.org/10.21015/vtse.v9i4.863

64. Varada Prashant Rao, Venugopal Shankar (2023) Accelerating the development of Software Defined Vehicles through Innovative Collaboration across the Industry. 2023 IEEE International Transportation Electrification Conference (ITEC-India). https://doi.org/10.1109/ITEC-India59098.2023.10471519

65. F. Pan, Jianjie Lin, Markus Rickert, Alois Knoll (2023) Automated Design Space Exploration for Resource Allocation in Software-Defined Vehicles. 2023 IEEE Intelligent Vehicles Symposium (IV). https://doi.org/10.1109/IV55152.2023.10186605

66. Christian Hackenbeck, Pascal Hirmer, Alexander Walz, et al (2025) Towards Standardized Interfaces for Software-Defined Vehicles in Automotive. 2025 IEEE International Automated Vehicle Validation Conference (IAVVC). https://doi.org/10.1109/IAVVC61942.2025.11219518

67. S. Oh, Y. Do, SeongHyeon Lim, et al (2025) Design of a Scaled-Vehicle Research Platform for Software-Defined Vehicles. International Conference on Mechatronics and Robotics Engineering. https://doi.org/10.1109/ICMRE64970.2025.10976315

68. J. Lee, Joo-Kyung Kim, S. Park, et al (2025) Software-Defined Product Architecture: Status, Challenges, and Future Perspectives. IEEE International Conference on Industrial Engineering and Engineering Management. https://doi.org/10.1109/IEEM63636.2025.11357662

69. Krzysztof Lebioda, Viktor Vorobev, Nenad Petrovic, et al (2024) Towards Single-System Illusion in Software-Defined Vehicles - Automated, AI-Powered Workflow. arXivorg. https://doi.org/10.48550/arXiv.2403.14460

70. Dusan Kenjic, Marija Antic (2023) Connectivity Challenges in Automotive Solutions. IEEE Consumer Electronics Magazine. https://doi.org/10.1109/MCE.2022.3183807

71. Yiwen Shen, Yoseop Ahn, Mose Gu, J. Jeong (2024) Intent-Based Management for Software-Defined Vehicles in Intelligent Transportation Systems. IEEE Conference on Network Softwarization. https://doi.org/10.1109/NetSoft60951.2024.10588911

72. Elena Lisova, Ruben Broux, J. Denil, et al (2022) Communication Patterns for Evaluating Vehicular E/E Architectures. 2022 International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME). https://doi.org/10.1109/ICECCME55909.2022.9987747

73. Abir Bazzi, Adnan Shaout, Di Ma (2024) A Novel Variability-Rich Scheme for Software Updates of Automotive Systems. IEEE Access. https://doi.org/10.1109/ACCESS.2024.3409629

74. Guoqi Xie, Wei Wu, Gang Zeng, et al (2020) Risk Assessment and Development Cost Optimization in Software Defined Vehicles. IEEE transactions on intelligent transportation systems (Print). https://doi.org/10.1109/TITS.2020.3027469

75. Gerd Winkler (2021) The Car's Electronic Architecture in Motion: The Coming Transformation

76. Vaibhav Saini, Ayushi Jain, Pramoda Meduri, Verolt Technology Solutions GmbH (2025) Revolutionizing Commercial Vehicles: Integrating Software-Defined Vehicle Architecture for Enhanced Electrical and Electronic Systems. SAE technical paper series. https://doi.org/10.4271/2025-01-8138

77. Patrizio Pelliccione, E. Knauss, S. Ågren, et al (2020) Beyond connected cars: A systems of systems perspective. Science of Computer Programming. https://doi.org/10.1016/j.scico.2020.102414

78. F. Pan, Jianjie Lin, Markus Rickert (2024) Automatic Platform Configuration and Software Integration for Software-Defined Vehicles. Information Security Solutions Europe. https://doi.org/10.1109/ISSE65546.2025.11370099

79. M. Spencer, M. Andreozzi (2023) Real-Time Portability in Software-Defined Systems. CPS-IoT Week Workshops. https://doi.org/10.1145/3576914.3587504

80. Akshay Mathur, Amit Misra, Sandeep Makam (2026) The Road to SDV: A Maturity Model for Software-Defined Transformation. SAE technical paper series. https://doi.org/10.4271/2026-26-0679

81. Misbah Ullah Khan, Vishal Gupta (2026) Achieving Reliability, Safety & Security in SDV OS Architecture. SAE technical paper series. https://doi.org/10.4271/2026-28-0122

82. Marc Schindewolf, Daniel Grimm, Christian Lingor, E. Sax (2022) Toward a Resilient Automotive Service-Oriented Architecture by using Dynamic Orchestration. 2022 IEEE 1st International Conference on Cognitive Mobility (CogMob). https://doi.org/10.1109/CogMob55547.2022.10118016

83. Luca Cuomo, Claudio Scordino, A. Ottaviano, et al (2023) Towards a RISC-V Open Platform for Next-generation Automotive ECUs. Mediterranean Conference on Embedded Computing. https://doi.org/10.1109/MECO58584.2023.10154913

84. Abir Bazzi, Adnan Shaout, Di Ma (2024) Software Update Methodologies for Feature-Based Product Lines: A Combined Design Approach. Software. https://doi.org/10.3390/software3030017

85. F. Pan, Jianjie Lin, Markus Rickert, Alois Knoll (2022) Resource Allocation in Software-Defined Vehicles: ILP Model Formulation and Solver Evaluation. 2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC). https://doi.org/10.1109/ITSC55140.2022.9922526

86. V. Bandur, Ryan Kapinski, Vera Pantelic, et al (2022) Aspects of Migrating from Decentralized to Centralized E/E Architectures. SAE technical paper series. https://doi.org/10.4271/2022-01-0747

87. M. Rotaru, K. Fukuoka, S. Otani, et al (2025) Forum 6: Evolution of the Software-Defined Vehicle: Navigating Smart Cockpits and In-Car Hardware. IEEE International Solid-State Circuits Conference. https://doi.org/10.1109/ISSCC49661.2025.10904587

88. N. Ferreira, Miguel Luís, P. Rito, et al (2024) Multihoming in Software Defined Vehicular Networks. IEEE Vehicular Networking Conference. https://doi.org/10.1109/VNC61989.2024.10575942

89. T. Häckel, Anja Schmidt, Philipp Meyer, et al (2020) Strategies for Integrating Controls Flows in Software-Defined In-Vehicle Networks and Their Impact on Network Security. arXivorg

90. Rodi Jolak, Vard Antinyan, Alexander Åström, et al (2026) Navigating the Future: Essential Considerations for the Engineering of Software-Defined Vehicles. Computer. https://doi.org/10.1109/MC.2025.3580508

91. Kim Strandberg, Ulf Arnljung, T. Olovsson, D. Oka (2023) Secure Vehicle Software Updates: Requirements for a Reference Architecture. IEEE Vehicular Technology Conference. https://doi.org/10.1109/VTC2023-Spring57618.2023.10199410

92. Bálint Máté, Max Scheerer, Ralf Sieger, et al (2025) Software-Defined Vehicles: Bridging Industry and Research Perspectives. International Symposium on Industrial Electronics. https://doi.org/10.1109/ISIE62713.2025.11124637

93. Kim Strandberg, D. Oka, T. Olovsson (2021) UniSUF: a unified software update framework for vehicles utilizing isolation techniques and trusted execution environments. https://doi.org/10.13154/294-8353

94. Markus Glaser, C. Macfarlane, B. May, et al (2020) Open standards enable continuous software development in the automotive industry

95. Boris Boehlen, Diana Fischer, Jue Wang (2024) Diagnostics of Automotive Service-Oriented Architectures with SOVD. SAE technical paper series. https://doi.org/10.4271/2024-01-7036

96. Sandra Bickelhaupt, Michael Hahn, N. Nuding, et al (2023) Challenges and Opportunities of Future Vehicle Diagnostics in Software-Defined Vehicles. SAE technical paper series. https://doi.org/10.4271/2023-01-0847

97. Ryo Kurachi (2025) Security Issues and Evolution in Software-Defined Vehicle (SDV). IEICE ESS FUNDAMENTALS REVIEW. https://doi.org/10.1587/essfr.19.2_87

98. Yunje Shin, Sanghoon Jeon (2024) MQTree: Secure OTA Protocol Using MQTT and MerkleTree. Italian National Conference on Sensors. https://doi.org/10.3390/s24051447

99. Dusan Kenjic, Dusan Zivkov, Marija Antic (2023) Automated Data Transfer From ADAS to Android-Based IVI Domain Over SOME/IP. IEEE Transactions on Intelligent Vehicles. https://doi.org/10.1109/TIV.2023.3236581

100. Zachariah Threet, C. Papadopoulos, W. Lambert, et al (2022) Securing Automotive Architectures with Named Data Networking. 2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC). https://doi.org/10.1109/ITSC55140.2022.9922194

101. Krish Agrawal, Rohan Jha, Nishkarsh Luthra, et al (2025) Advancing Software-Defined Vehicles: An End-to-End Framework with Digital Twin Based Attestation for OTA Updates. International Conference on Communication Systems and Networks. https://doi.org/10.1109/COMSNETS63942.2025.10885560

102. Christian Prehofer (2025) Self-Sovereign Identities for Software-Defined Vehicles. 2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW). https://doi.org/10.1109/EuroSPW67616.2025.00043

103. Jaewan Seo, Jiwon Kwak, Seungjoo Kim (2024) Development for High-Assurance Software Update Management System Complying With UN R156. IEEE Access. https://doi.org/10.1109/ACCESS.2024.3426591

104. Bahar Houtan, Mehmet Onur Aybek, M. Ashjaei, et al (2022) End-to-end Timing Model Extraction from TSN-Aware Distributed Vehicle Software. EUROMICRO Conference on Software Engineering and Advanced Applications. https://doi.org/10.1109/SEAA56994.2022.00064

105. M. Haeberle, Florian Heimgaertner, H.-G. Loehr, et al (2020) An SDN Architecture for Automotive Ethernets. https://doi.org/10.15496/PUBLIKATION-41962

106. Yichen Luo, Sidi Lu (2025) Incremental Over-the-Air Update for Software-Defined Vehicles with C-V2X. 2025 IEEE 101st Vehicular Technology Conference (VTC2025-Spring). https://doi.org/10.1109/VTC2025-Spring65109.2025.11174478

107. T. Häckel, Anja Schmidt, Philipp Meyer, et al (2020) Strategies for Integrating Control Flows in Software-Defined In-Vehicle Networks and Their Impact on Network Security. IEEE Vehicular Networking Conference. https://doi.org/10.1109/VNC51378.2020.9318372

108. H. Askaripoor, Thilo Mueller, Alois C. Knoll (2024) E/E Designer: A Framework to Design and Synthesize Vehicle E/E Architecture. IEEE Transactions on Intelligent Vehicles. https://doi.org/10.1109/TIV.2023.3324617

109. Katharina Polanec, Jounes-Alexander Gross, Christian Neureiter (2024) A Domain-Specific Modeling Framework for the Development of Automotive Systems. IEEE International Conference on Emerging Technologies and Factory Automation. https://doi.org/10.1109/ETFA61755.2024.10711101

110. Youssef Elkharaz, Saad Motahhir, A. E. Ghzizal (2021) Comparison between autosar platforms with functional safety for automotive software architectures. arXivorg

111. V. Mosin, Darko Durisic, M. Staron (2021) Applicability of Machine Learning Architectural Patterns in Vehicle Architecture: A Case Study. European Conference on Software Architecture

112. T. Anderson, Scott Stevens, Jörg Schäuffle (2025) Rapidly Evolving E/E Architectures for Ground Vehicles - Insights from the Commercial and Automotive Industries. SAE technical paper series. https://doi.org/10.4271/2025-01-0446

113. Alex Richardson, M. Nice, Matt Bunting, et al (2023) Analysis of a Runtime Data Sharing Architecture over LTE for a Heterogeneous CAV Fleet. CPS-IoT Week Workshops. https://doi.org/10.1145/3576914.3587712

114. S. S, Kumar P, R. M. Suchindhar, Karupannan P (2024) Enhanced Software Deployment and Integration with an Integrated CI/CD Pipeline. 2024 International Conference on Smart Technologies for Sustainable Development Goals (ICSTSDG). https://doi.org/10.1109/ICSTSDG61998.2024.11026461

115. J. Heidrich, Michael Kläs, A. Morgenstern, et al (2021) From Complexity Measurement to Holistic Quality Evaluation for Automotive Software Development. arXivorg

116. Patrick Schmidt, Iuliia Topko, Matthias Stammler, et al (2024) EMDRIVE Architecture: Embedded Distributed Computing and Diagnostics from Sensor to Edge. Design, Automation and Test in Europe. https://doi.org/10.23919/DATE58400.2024.10546796

117. Elisa Calisesi, Ozlem Kaya, Andrea Melis, R. Girau (2024) A Federated Architecture for Digital Twins in Automotive: An Integrated Solution for Vehicles, Drivers and Road Environments. Conference on Information Technology for Social Good. https://doi.org/10.1145/3677525.3678659

118. Amitav Swapnil, Lasse Metso, Kameswara Satya Prakash Oruganti, Ari Happonen (2024) Driving the Future: Utilizing Software Foundations to Improve Vehicles Functionalities. 2024 4th International Conference on Electrical, Computer, Communications and Mechatronics Engineering (ICECCME). https://doi.org/10.1109/ICECCME62383.2024.10796847

119. Christian Hackenbeck, Alexander Walz, Pascal Hirmer, et al (2025) Semantic Interface Modeling for Automotive Architectures Using a Domain-Driven Approach. IFAC-PapersOnLine. https://doi.org/10.1016/j.ifacol.2025.11.936

120. Yuchu Liu, J. Bosch, H. H. Olsson, Jonn Lantz (2021) An architecture for enabling A/B experiments in automotive embedded software. Annual International Computer Software and Applications Conference. https://doi.org/10.1109/COMPSAC51774.2021.00134

121. Nenad Petrovic, Vahid Zolfaghari, F. Pan, Alois Knoll (2026) LLM-Empowered Functional Safety and Security by Design in Automotive Systems. arXivorg. https://doi.org/10.48550/arXiv.2601.02215

122. Jan Ruhnau, Martin Sommer, Jacqueline Henle, et al (2023) Ontology for Vehicle Function Distribution. IEEE Systems Conference. https://doi.org/10.1109/SysCon53073.2023.10131097

123. Ronak Indrasinh Kosamia (2021) A Microkernel-Inspired Modular Framework For Fault-Isolated Multi-Domain IVI Services In Android-Based Software-Defined Vehicles. International Journal for Sciences and Technology. https://doi.org/10.71097/ijsat.v12.i1.8116

124. Akshay Narla, Daniel Dittler, Nasser Jazdi, M. Weyrich (2025) A Framework for Interoperability in Software Defined Vehicles using Asset Administration Shells. Industrial Cyber-Physical Systems. https://doi.org/10.1109/ICPS65515.2025.11087876

125. J. Blech (2020) Towards Digital Twins for the Description of Automotive Software Systems. QAPL. https://doi.org/10.4204/EPTCS.312.2

126. X. Yang (2024) Framework of Electric Vehicle Fault Diagnosis System Based on Diagnostic Communication. International Journal of Engineering. https://doi.org/10.5829/ije.2024.37.06c.16

127. Jaewan Seo, Jiwon Kwak, Seungjoo Kim (2023) Formally Verified Software Update Management System in Automotive. VehicleSec. https://doi.org/10.14722/vehiclesec.2023.23087

128. Alexander Stocker, Tahir Emre Kalayci, Michael Spitzer, Gerald Musser (2025) Context-Aware Warning Systems: Leveraging Driving Environment Data for Improved Driver and Road User Warnings. International Conference on Web Information Systems and Technologies. https://doi.org/10.5220/0013820600003985

129. J. Sobotka, Jirí Novák, Jiří Pinkava (2023) Rapid Prototyping of Vehicle Software Defined Functions. Proceedings of the 26th IMEKO TC4 International Symposium and 24th International Workshop on ADC/DAC Modelling and Testing. https://doi.org/10.21014/tc4-2023.20

130. Raphael Weber, N. Adler, Thomas Wilhelm, et al (2022) Towards Automating a Software-Centered Development Process that considers Timing Properties. ACM Symposium on Cloud Computing. https://doi.org/10.1109/SOCC56010.2022.9908127

131. Aniello Castiglione, Lucia Cimmino, Michele Nappi, Luigi Emanuele Sica (2025) OpenV2X: A Modular Cyber-Physical Framework for Vision-Driven Environmental Perception and Interactive Feedback in Software-Defined Vehicles. IEEE Transactions on Industrial Informatics. https://doi.org/10.1109/tii.2025.3641528

132. Ajit Gajre (2025) Driveline Control Integrated with Zonal Architectures in Software-Defined Vehicles. International Journal of Computational and Experimental Science and Engineering. https://doi.org/10.22399/ijcesen.4110

133. Dirk Slama, Jens Lachenmaier (2024) Enabling applications for variable vehicle configurations with Software-defined Vehicle and Service-oriented Architecture. Procedia CIRP. https://doi.org/10.1016/j.procir.2024.08.341

134. Jacqueline Henle, S. Otten, Eric Sax (2024) Systems Engineering Approach for Compliant Over-the-Air Update Development. 2024 IEEE International Conference on Recent Advances in Systems Science and Engineering (RASSE). https://doi.org/10.1109/RASSE64357.2024.10773922

135. Guilherme A. Thomaz, Thierno Barry, Matteo Sammarco, M. Campista (2025) End-to-end trusted computing architecture for vehicular over-the-air updates. Annals of Telecommunications. https://doi.org/10.1007/s12243-025-01096-y